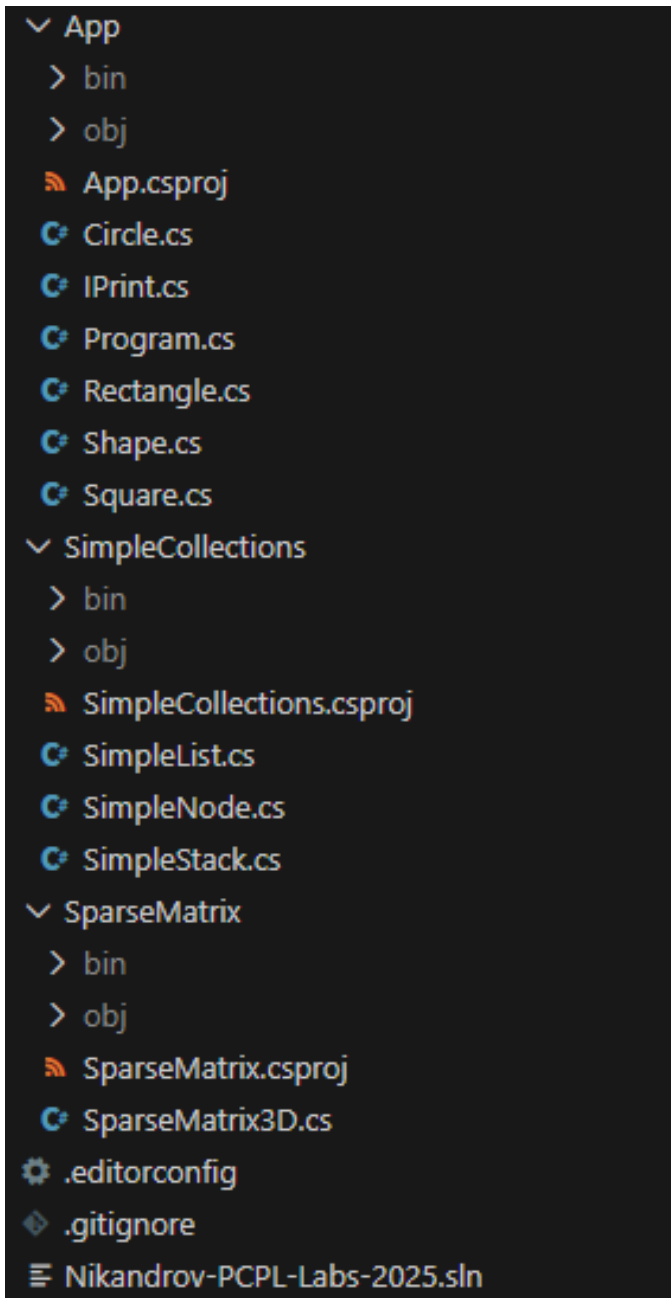


Отчет по лабораторной работе №3 по дисциплине ПиКЯП

Никандров Валентин ИУ5-35Б

Файловая структура проекта:



Листинг кода проекта:

```
using System.Collections;

using Shapes;
using SparseMatrix;
using SimpleCollections;

class Program
{
    static void Main(string[] args)
    {
        ArrayList arrayList = new();
        arrayList.Add(new Rectangle(2, 3));
        arrayList.Add(new Square(2));
        arrayList.Add(new Circle(1));

        Console.WriteLine("ArrayList: ");
        PrintCollection(arrayList);

        arrayList.Sort();

        Console.Write('\n');
        PrintCollection(arrayList);

        List<Shape> list = new();
        list.Add(new Rectangle(2, 3));
        list.Add(new Square(2));
        list.Add(new Circle(1));

        Console.WriteLine("\nList:");
        PrintCollection(list);

        list.Sort();

        Console.Write('\n');
        PrintCollection(list);

        Console.WriteLine("\nSparseMatrix:");
        SparseMatrix3D<Shape> sparseMatrix3D = new(3, 3, 3);
        sparseMatrix3D[1, 1, 1] = new Circle(1);

        Console.WriteLine(sparseMatrix3D.ToString());

        SimpleStack<Shape> stack = new();
        stack.Push(new Circle(1));

        Console.WriteLine("\nStack:");
        Console.WriteLine(stack.Pop().ToString());
    }
}
```

```

private static void PrintCollection(ArrayList arrayList)
{
    foreach (var element in arrayList)
    {
        Console.WriteLine(element.ToString());
    }
}

private static void PrintCollection(List<Shape> list)
{
    foreach (Shape shape in list)
    {
        Console.WriteLine(shape.ToString());
    }
}
}

```

```

namespace Shapes;

public abstract class Shape : IComparable
{
    public virtual double CalculateArea() => 0.0;

    public int CompareTo(object? obj)
    {
        if (obj is Shape shape) return
CalculateArea().CompareTo(shape.CalculateArea());
        else throw new ArgumentException("Argument is incomparable");
    }
}

```

```

public interface IPrint
{
    public void Print();
}

```

```

namespace Shapes;

public class Rectangle : Shape, IPrint
{
    public Rectangle(double width, double height)
    {
        Width = Math.Abs(width);
        Height = Math.Abs(height);
    }
}

```

```

    public double Width { get; init; }
    public double Height { get; init; }

    public void Print() => Console.WriteLine(ToString());

    public override double CalculateArea() => Width * Height;

    public override string ToString() => $"Width: {Width:F4}; Height:
{Height:F4}; Area: {CalculateArea():F4}";
}

```

```

namespace Shapes;

public class Square : Rectangle
{
    public Square(double length) : base(length, length) { }

    public override string ToString() => $"Side length: {Width:F4}; Area:
{CalculateArea():F4}";
}

```

```

namespace Shapes;

public class Circle : Shape, IPrint
{
    public Circle(double radius)
    {
        Radius = Math.Abs(radius);
    }

    public double Radius { get; init; }

    public void Print() => Console.WriteLine(ToString());

    public override double CalculateArea() => Math.PI * Radius * Radius;

    public override string ToString() => $"Radius: {Radius:F4}; Area:
{CalculateArea():F4}";
}

```

```

namespace SparseMatrix;

public class SparseMatrix3D<T>
{

```

```

private Dictionary<(int, int, int), T?> _values = new();
private int _rowCount;
private int _columnCount;
private int _layersCount;

public SparseMatrix3D(int rowCount, int columnCount, int layersCount)
{
    if (rowCount < 0 || columnCount < 0 || layersCount < 0)
    {
        throw new ArgumentOutOfRangeException("Argument out of range");
    }

    _rowCount = rowCount;
    _columnCount = columnCount;
    _layersCount = layersCount;
}

public T? this[int row, int column, int layer]
{
    get
    {
        if (!ValidateCoords(row, column, layer))
        {
            throw new IndexOutOfRangeException("Index out of range");
        }

        if (_values.ContainsKey((row, column, layer)))
        {
            return _values[(row, column, layer)];
        }
        else
        {
            return default;
        }
    }

    set
    {
        if (!ValidateCoords(row, column, layer))
        {
            throw new IndexOutOfRangeException("Index out of range");
        }

        if (value is null)
        {
            _values.Remove((row, column, layer));
        }
        else
        {
            _values[(row, column, layer)] = value;
        }
    }
}

```

```

    }
}

public override string ToString()
{
    string result = "\n";

    foreach (var element in _values)
    {
        result += $"Coords: {element.Key.Item1} {element.Key.Item2}
{element.Key.Item3}; Value: {element.Value?.ToString() ?? "Null"}\n";
    }

    return result;
}

private bool ValidateCoords(int row, int column, int layer)
{
    return row >= 0 && row < _rowCount &&
        column >= 0 && column < _columnsCount &&
        layer >= 0 && layer < _layersCount;
}
}

```

```

namespace SimpleCollections;

public class SimpleNode<T>
{
    public SimpleNode(T value)
    {
        Value = value;
    }

    public T Value { get; set; }
    public SimpleNode<T>? Next { get; set; }
}

```

```

using System.Collections;

namespace SimpleCollections;

public class SimpleList<T> : IEnumerable<T>
{
    private SimpleNode<T>? _head;
    private SimpleNode<T>? _tail;
    public int Length { get; private set; }
}

```

```

public void Add(T value)
{
    SimpleNode<T> newNode = new(value);

    if (_head is null)
    {
        _head = newNode;
        _tail = newNode;
    }
    else
    {
        _tail!.Next = newNode;
        _tail = _tail.Next;
    }

    ++Length;
}

public bool Remove(T value)
{
    if (Length == 0) throw new InvalidOperationException("List is empty");

    SimpleNode<T>? current = _head;
    SimpleNode<T>? previous = null;

    while (current is not null)
    {
        if (current.Value is not null && current.Value.Equals(value))
        {
            if (previous is null)
            {
                _head = current.Next;

                if (_head is null)
                {
                    _tail = null;
                }
            }
            else
            {
                previous.Next = current.Next;

                if (current.Next is null)
                {
                    _tail = previous;
                }
            }

            --Length;
            return true;
        }
    }
}

```

```

        previous = current;
        current = current.Next;
    }

    return false;
}

public void RemoveAt(int index)
{
    if (Length == 0) throw new InvalidOperationException("List is empty");
    else if (index < 0 || index >= Length) throw new
IndexOutOfRangeException();

    SimpleNode<T>? current = _head;
    SimpleNode<T>? previous = null;

    for (int i = 0; i < index; i++)
    {
        previous = current;
        current = current!.Next;
    }

    if (previous is null)
    {
        _head = current!.Next;

        if (_head is null)
        {
            _tail = null;
        }
    }
    else
    {
        previous.Next = current!.Next;

        if (current.Next is null)
        {
            _tail = previous;
        }
    }

    --Length;
}

public T this[int index]
{
    get
    {
        if (index < 0 || index >= Length) throw new
IndexOutOfRangeException();
    }
}

```



```

        SimpleNode<T>? current = _head;

        for (int i = 0; i < index; i++)
        {
            current = current!.Next;
        }

        return current!.Value;
    }

    set
    {
        if (index < 0 || index >= Length) throw new
IndexOutOfRangeException();

        SimpleNode<T>? current = _head;

        for (int i = 0; i < index; i++)
        {
            current = current!.Next;
        }

        current!.Value = value;
    }
}

public bool Find(T value)
{
    SimpleNode<T>? current = _head;

    while (current is not null && current.Value is not null)
    {
        if (current.Value.Equals(value)) return true;
        current = current.Next;
    }

    return false;
}

IEnumerator<T> IEnumerable<T>.GetEnumerator()
{
    SimpleNode<T>? current = _head;

    while (current is not null)
    {
        yield return current.Value;
        current = current.Next;
    }
}

```

```
IEnumerator IEnumerable.GetEnumerator()  
{  
    return ((IEnumerable<T>)this).GetEnumerator();  
}  
  
}
```

```
namespace SimpleCollections;  
  
public class SimpleStack<T> : SimpleList<T>  
{  
    public void Push(T value)  
    {  
        Add(value);  
    }  
  
    public T Pop()  
    {  
        if (Length == 0) throw new InvalidOperationException("Stack is empty");  
  
        T value = this[Length - 1];  
        RemoveAt(Length - 1);  
        return value;  
    }  
}
```

Результат работы программы:

```
vIntester@DESKTOP-ACISLUE:~/Nikandrov-PCPL-Labs-2025/App$ dotnet run
ArrayList:
Width: 2.0000; Height: 3.0000; Area: 6.0000;
Side length: 2.0000; Area: 4.0000;
Radius: 1.0000; Area: 3.1416

Radius: 1.0000; Area: 3.1416
Side length: 2.0000; Area: 4.0000;
Width: 2.0000; Height: 3.0000; Area: 6.0000;

List:
Width: 2.0000; Height: 3.0000; Area: 6.0000;
Side length: 2.0000; Area: 4.0000;
Radius: 1.0000; Area: 3.1416

Radius: 1.0000; Area: 3.1416
Side length: 2.0000; Area: 4.0000;
Width: 2.0000; Height: 3.0000; Area: 6.0000;

SparseMatrix:

Coords: 1 1 1; Value: Radius: 1.0000; Area: 3.1416

Stack:
Radius: 1.0000; Area: 3.1416
```